



Anil Seth

Plone with Schemas

Using Dexterity to extend and customise Plone.

Last month, you tried Plone 4 and could create a schema interactively. However, at present, it does not seem to be possible to convert the interactive definitions into code. So, this month we will explore how to add code for two related schemas, and create a simple custom view to display them.

Just as with Grok, the framework will create the skeleton package, so that the routine drudgery is removed. This is done by *zopeskel.dexterity* (see <http://plone.org/products/zopeskel.dexterity> for details).

You may wish to install Plone as a non-root user, in which case, the installation is in *\$HOME/Plone*. The advantage, of course, is that you will not need to be a super-user for development activities. Once Plone 4 is installed, your next step is to modify the build-out configuration files, to add needed components. *Zopeskel* is defined in *base.cfg*. You will need to add the egg for *zopeskel.dexterity* and the required dependencies. Your *zopeskel* section in *base.cfg* should now look like what is shown below:

```
[zopeskel]
# installs paster and Zopeskel
recipe = zc.recipe.egg
eggs =
    PasteScript
    ZopeSkel
    Paste
    PasteDeploy
    zopeskel.dexterity
$(buildout:eggs)
```

You should change the *extends* section of *buildout.cfg* to look like what follows:

```
extends =
    base.cfg
    versions.cfg
http://good-py.appspot.com/release/dexterity/2.0-next
```

You will also need to include the egg for Dexterity in your Plone instance. The *eggs* section in your *buildout.cfg* will look like what follows:

```
eggs =
    Plone
```

```
Pillow
lxml
plone.app.dexterity
```

Now, you may run *buildout* to update the installation:

```
$ bin/buildout
```

You can verify that the site works as it did before.

Creating schemas

Move to the *src* sub-directory and create the skeleton as follows. You will be prompted to specify a package name, e.g., *lfy.demo*. A sub-directory by the same name will be created by *zopeskel*.

```
$ ../bin/zopeskel dexterity
$ cd lfy.demo
```

You may want to create two related schemas. For example, 'Department' and 'Writer'; a writer belongs to a department. You will need to run the *paster* command for each content type and you will be prompted for the name of the content type (that is, the schema).

```
$ ../bin/paster addcontent dexterity_content
$ ../bin/paster addcontent dexterity_content
```

Directory structure and skeleton files, including Python code, will be created, which you can expand.

Now, you will want the department to have a name and description of its responsibilities. In addition, you may want to have the photo of the head of the department. Regarding the writer, you would require his name, blog site, photo, a brief profile and the department to which he belongs. You can define the models in the Python files in the sub-directory *lfy.demo*. The generated skeleton files are well documented. The *IDepartment* class in *department.py* will then become what's given below:

```
from zope import schema
from plone.app.textfield import RichText
from plone.namedfile.field import NamedImage
```